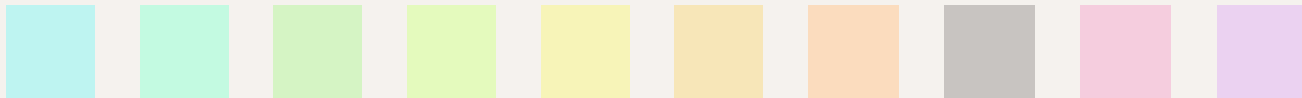


Client Side CSS

מבוא לבניית מערכות ממוחשבות מבוססות אינטרנט הרצאה 3 – CSS





Transitions

CSS Transitions

- CSS transitions allows you to change property values smoothly, over a given duration.
- Relevant properties:
 1. transition
 2. transition-delay
 3. transition-duration
 4. transition-property
 5. transition-timing-function

How to Use CSS Transitions?

- To create a transition effect, you must specify two things:
 1. the CSS property you want to add an effect to
 2. the duration of the effect
- Note: If the duration part is not specified, the transition will have no effect, because the default value is 0.
- Example:

```
div {  
  width: 100px;  
  height: 100px;  
  background: red;  
  transition: width 2s;  
}
```

```
div:hover {  
  width: 300px;  
  Height: 200px;  
}
```

CSS Transitions Properties

Property	Description
transition	A shorthand property for setting the four transition properties into a single property (property, duration, timing function, delay)
transition-delay	Specifies a delay (in seconds) for the transition effect
transition-duration	Specifies how many seconds or milliseconds a transition effect takes to complete
transition-property	Specifies the name of the CSS property the transition effect is for
transition-timing-function	Specifies the speed curve of the transition effect



Animations

CSS Animations

- CSS allows animation of HTML elements without using JavaScript or Flash

What are CSS Animations?

- An animation lets an element gradually change from one style to another.
- You can change as many CSS properties you want, as many times you want.
- To use CSS animation, you must first specify some keyframes for the animation.
- Keyframes hold what styles the element will have at certain times.

CSS Animations Relevant Properties

1. animation-name
2. animation-duration
3. animation-delay
4. animation-iteration-count
5. animation-direction
6. animation-timing-function
7. animation

CSS Animations Relevant Properties

1. **animation-name**: declares the name of the @keyframes at-rule to manipulate.

CSS Animations Relevant Properties

2. **animation-duration**: the length of time it takes for an animation to complete one cycle.

CSS Animations Relevant Properties

- 3. **animation-timing-function**: establishes preset acceleration curves such as ease or linear.

The `animation-timing-function` property specifies the speed curve of the animation.

CSS Animations Relevant Properties

4. `animation-delay`: the time between the element being loaded and the start of the animation sequence.

The `animation-delay` property specifies a delay for the start of an animation (*negative or positive number of seconds*).

CSS Animations Relevant Properties

- 5. **animation-direction**: sets the direction of the animation after the cycle. Its default resets on each cycle.

The `animation-direction` property specifies whether an animation should be played forwards, backwards or in alternate cycles.

CSS Animations Relevant Properties

6. animation-iteration-count: the number of times the animation should be performed.

The `animation-iteration-count` property specifies the number of times an animation should run (*number* or *'infinite'*).

The @keyframes Rule

- When you specify CSS styles inside the @keyframes rule, the animation will gradually change from the current style to the new style at certain times.
- To get an animation to work, you must bind the animation to an element.

The @keyframes Rule

- The following example binds the "example" animation to the <div> element. The animation will last for 4 seconds, and it will gradually change the background-color of the <div> element from "red" to "yellow"



```
/* The animation code */
@keyframes example {
  from {background-color: red;}
  to {background-color: yellow;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```

```
/* The animation code */
@keyframes example {
  0%   {background-color: red;}
  25%  {background-color: yellow;}
  50%  {background-color: blue;}
  100% {background-color: green;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```

Transitions VS Animations

Transitions	Animations
Initial to final state- no intermediate steps	Initial to final state- with intermediate steps
Can only run once	Can run multiple times
Requires a trigger to run	Can be triggered or run automatically
Run forward with trigger and reverse when trigger is removed	Can run forward, reverse or alternate directions



Responsive Web Design (RWD)

Responsive Web Design (RWD)

- RWD is about creating web pages that look good on all devices!
- A RWD will automatically adjust for different screen sizes and viewports (the visible area of the users' browser's screen).
- RWD is accomplished using:
 - Media queries
 - Flexbox
 - Grid
 - bootstrap



(1) RWD- Media Queries

Media Queries

- Media queries are CSS3 extensions to media types that gives us more control over rendering across different devices.
- MQ will allow us to change our layouts to suit the exact need of different devices
- The use of MQ doesn't require us to change the content

Responsive Web Design with @media



Media Queries

What is a Media Query?

- A MQ is a logical expression that is either True or False.
- It uses the @media rule to include a block of CSS properties only if a certain condition is true.

Media Queries - Definition and Usage

- The @media rule is used to apply different styles for different media types/devices.
- Media queries can be used to check many things, such as:
 1. width and height of the device
 2. Orientation- portrait or landscape
 3. Etc.

Media Query Syntax- usually used

```
@media (expressions) {  
    CSS-Code;  
}
```

```
@media not|only mediatype and (expressions) {  
    CSS-Code;  
}
```

@media Example

- If the browser window is 600px or smaller, the background color will be changing color:

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Different background color for different devices:

Here we are setting 2 @media rules

```
body {  
    background-color: aquamarine;  
}  
  
@media (min-width:319px) and (max-width:480px){  
    body {  
        background-color: lightsalmon;  
    }  
}  
  
@media (min-width:481px) and (max-width:1200px){  
    body {  
        background-color: powderblue;  
    }  
}  
  
@media (min-width:1201px) and (max-width:1600px){  
    body {  
        background-color: limegreen;  
    }  
}  
  
@media (min-width:1601px){  
    body {  
        background-color: seagreen;  
    }  
}
```

Screen Sizes

Most Common Desktop Screen Sizes

1366×768–23.49%
1920×1080–19.91%
1536×864–8.65%
1440×900–7.38%
1280×720–4.89%
1600×900–4.01%
1280×800–3.33%

Most Common Tablet Screen Sizes

768×1024–51.98%
1280×800–7.11%
800×1280–5.34%
601×962–4.47%
600×1024–2.85%
1024×1366–1.96%

Most Common Mobile Screen Sizes

360×640–17.91%
375×667–7.61%
414×896–6.52%
360×780–5.56%
360×760–5.06%
414×736–3.74%

With @media You Can also...

- Hide elements
- Change font size
- Etc...



(2) RWD- Flexbox

Flexbox

- The Flexible Box Layout Module makes it easier to design flexible responsive layout structure.
- Flexbox allows us to define 1D flexible container.

The flex Container

- To start using the Flexbox model, you need to first define a flex container (usually, a div).



- The element above represents a flex container (the violet area) with three flex items.
- The flex container becomes flexible by setting the display property to `display:flex`.

The flex Container



HTML:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
</div>
```

CSS:

```
.flex-container {  
  display: flex;  
  background-color: violet;  
}
```

The flex Container Properties

- The flex container properties are:
 - flex-direction
 - flex-wrap
 - flex-flow
 - justify-content
 - align-items
 - align-content
- <https://appbrewery.github.io/flex-layout/>

The flex Container Properties

- **flex-direction**: defines in which direction the container wants to stack the flex items (row \ column \ row-reverse \ column-reverse)
- **flex-wrap**: specifies whether the flex items should wrap or not (wrap \ nowrap \ wrap-reverse)
- **flex-flow**: shorthand for both flex-direction and flex-wrap
- **justify-content**: used to align the flex items horizontally.
- **align-items**: used to align the flex items vertically.
- **align-content**: used to align the flex lines (works only if flex-wrap:wrap). Defines how the items will be wrapped.

The flex Items

- The direct child elements of a flex container automatically becomes flexible (flex) items.

1

2

3

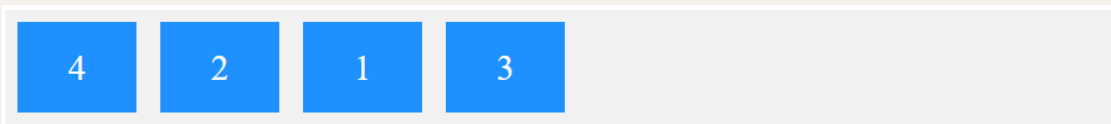
The flex item Properties

- The flex item properties are:
 - order
 - flex-grow
 - flex-shrink
 - flex-basis
 - flex
 - align-self

The flex item Properties

- **order**: specifies the order of the flex items (integer). The first flex item in the code does not have to appear as the first item in the layout.

```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```

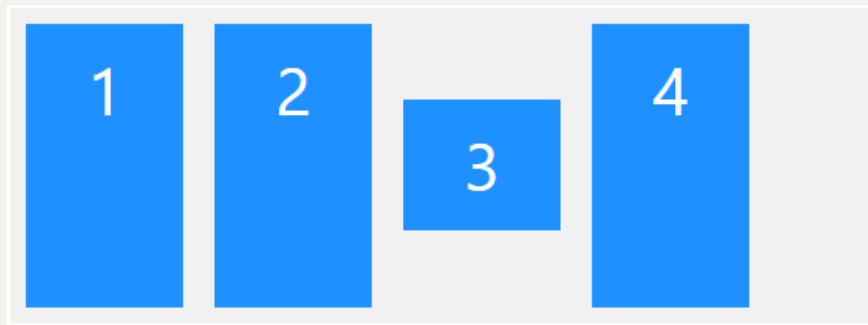


- **flex-grow**: specifies how much a flex item will grow relative to the rest of the flex items.
- **flex-shrink**: specifies how much a flex item will shrink relative to the rest of the flex items.

The flex item Properties

- **flex-basis**: specifies the initial length of a flex item (the size of it).
- **align-self**: specifies the alignment for the selected item inside the flexible container. It overrides the default alignment set by the container's align-items property.

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="align-self: center">3</div>  
  <div>4</div>  
</div>
```



Flexbox and Media Query

- Media query can also change the flexbox layout according to a specified condition.
- Example on VS code- Media-query+flex



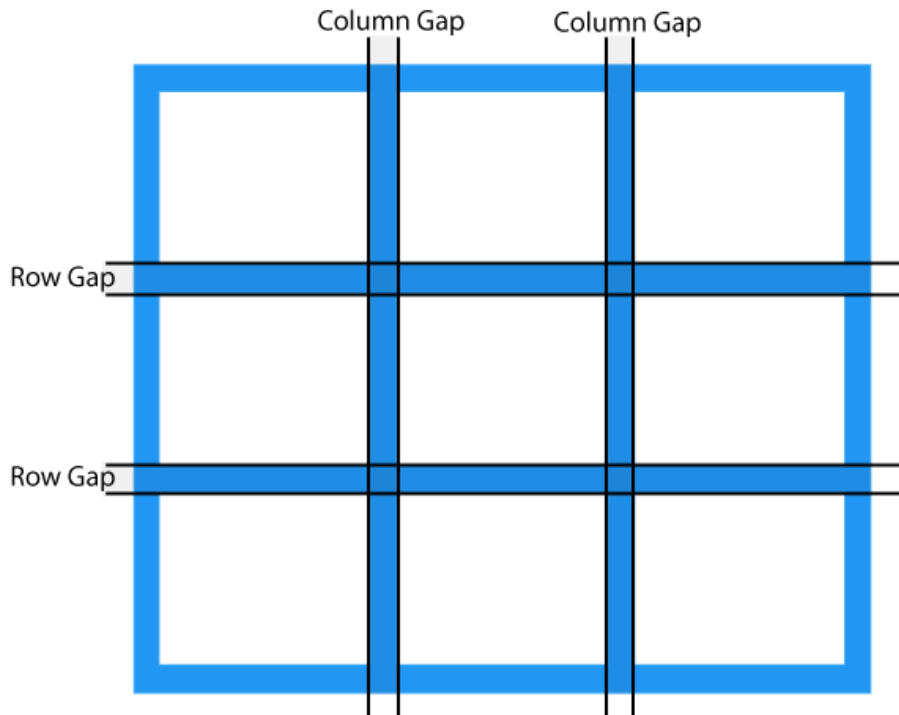
(3) RWD- Grid

Grid Layout Module

- The CSS Grid Layout Module offers a grid-based layout system, with rows and columns.

Grid Elements

- A grid layout consists of a parent element, with one or more child elements.
 - I. Grid container
 - II. Grid item
 - Grid cols
 - Grid rows
 - Grid gap



Grid Layout Module



Grid Container- Display Property

- An HTML element becomes a grid container when its display property is set to grid or inline-grid.

```
.grid-container {  
  display: grid;  
}
```

or

```
.grid-container {  
  display: inline-grid;  
}
```

- Inline-grid element is smaller and will fit to size of content.
 - Grid is larger and will fit the entire screen's width
- All direct children of the grid container automatically become *grid items*.

The Grid Container properties

- **grid-template-columns:** defines the number of columns in your grid layout, and it can define the width of each column.
- **grid-template-rows:** defines the number of rows in your grid layout, and it can define the width of each row.
- **justify-content:** used to horizontally align the whole grid inside the container.
- **align-content:** used to vertically align the whole grid inside the container.

Defining Rows' & Columns' Size

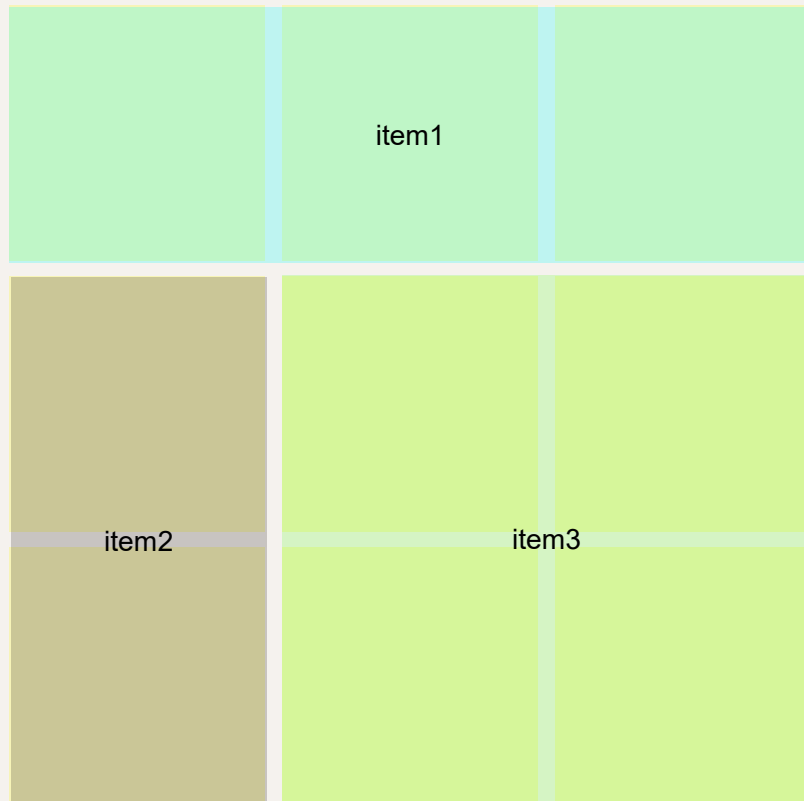
- The size of the rows and columns is defined using the grid-template-columns and grid-template-rows properties.
 - Fixed size (not responsive):
 - `grid-template-rows: 100px 100px;`
 - `grid-template-columns: 100px 100px;`
 - `/*2 rows and 2 columns, each 100px width and height*/`
 - Auto size:
 - `grid-template-rows: 100px auto;`
 - `grid-template-columns: 100px auto;`
 - `/*auto row will fit to the size of content; auto column will fit to the size of screen*/`

Defining Rows' & Columns' Size

- Fractional ratio size (fr):
 - `grid-template-rows: 1fr 2fr;`
 - `grid-template-columns: 1fr 2fr;`
 - `/*2nd row and 2nd column are twice the size of 1st row and 1st column*/`
- Minimax size:
 - `grid-template-rows: 200px 400px;`
 - `grid-template-columns: 200px minmax(400px,800px);`
 - `/*2nd column can grow up to 800px and shrink down to 400px*/`
- repeat size:
 - `grid-template-rows: repeat(2,200px);`
 - `grid-template-columns: repeat(2,200px);`
 - `/*2 rows and 2 columns, each 200px*/`

Grid items

- A grid *container* contains grid *items*.
- By default, a container has one grid item for each column, in each row, but you can style the grid items so that they will span multiple columns and/or rows.



The Grid item properties

- **grid-column**: defines on which column(s) to place an item.
 - `grid-column: 1 / 5`; (start on 1st column, end before 5th column)
 - `grid-column: 1 / span 4`; (start on 1st column, span 4 columns)

1				2	3
4	5	6	7	8	9
10	11	12	13	14	15

- **grid-row**: defines on which row to place an item.
 - `grid-row: 1 / 4`; (start on row-line 1 and end on row-line 4)
 - `grid-row: 1 / span 3`; (start on row 1 and span 3 rows)

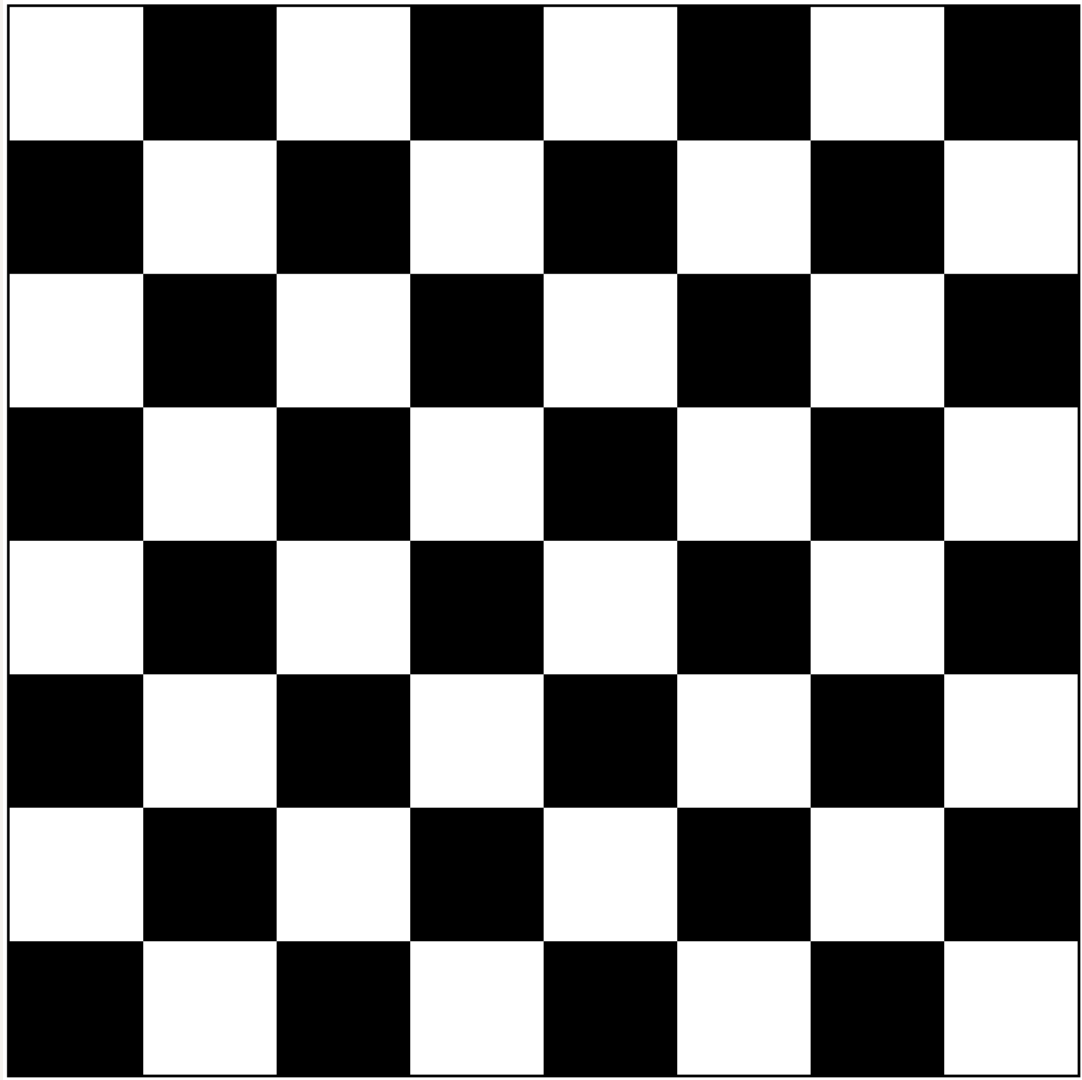
1	2	3	4	5	6
	7	8	9	10	11
	12	13	14	15	16

The Grid item properties

- **grid-area**: a shorthand property for the grid-row-start, grid-column-start, grid-row-end and the grid-column-end properties.
 - **grid-area**: 1 / 2 / 5 / 6;
 - (start on row-line 1 and column-line 2, and end on row-line 5 and column line 6)

1	8				2
3					4
5					6
7					9
10	11	12	13	14	15

Grid Ches board example





RWD- Bootstrap

RWD Frameworks → Bootstrap

- Another popular CSS framework is Bootstrap. Bootstrap uses HTML, CSS and jQuery to make responsive web pages.



Bootstrap

What is Bootstrap?

- Bootstrap is a free front-end framework for faster and easier web development
- Bootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables, navigation, modals, image carousels and many other, as well as optional JavaScript plugins
- Bootstrap also gives you the ability to easily create responsive designs

Easiest way to implement

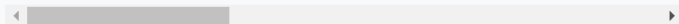
- On the <head> element, include a link using CDN (Content delivery network) to bootstrap:
 - <https://getbootstrap.com/>



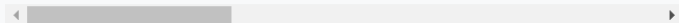
Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use [jsDelivr](#). See it in action with our simple [quick start](#), or [browse the examples](#) to jumpstart your next project. You can also choose to include Popper and our JS [separately](#).

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/b
```



```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/b
```



Using Bootstrap components

- For using Bootstrap components, add in the last row of the `<body>`:
 - `<script`
`src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-`
`C6RzsynM9kWDrMNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qy`
`q46cDfL" crossorigin="anonymous"></script>`





Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use [jsDelivr](#). See it in action with our simple [quick start](#), or [browse the examples](#) to jumpstart your next project. You can also choose to include Popper and our JS [separately](#).

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/b
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/b
```

RWD using Bootstrap

- Bootstrap include pre-designed classes for containers with a 12-column layout.

```
<div class="container">  
  <div class="row">  
    <div class="col-md-4 col-sm-12">col-4</div>  
    <div class="col-md-4 col-sm-12">col-4</div>  
    <div class="col-md-4 col-sm-12">col-4</div>  
  </div>  
</div>
```

- Div container → Row container → cloumns
- Split each row to 12 columns and span the content as you wish
- Use col-XXX classes to create responsive layouts

RWD using Bootstrap

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320px
<code>.container-lg</code>	100%	100%	100%	960px	1140px	1320px
<code>.container-xl</code>	100%	100%	100%	100%	1140px	1320px
<code>.container-xxl</code>	100%	100%	100%	100%	100%	1320px
<code>.container-fluid</code>	100%	100%	100%	100%	100%	100%

Exercise:

- Go to <https://appbrewery.github.io/bootstrap-layout/>
- Your goal- the bottom container should behave as the upper container.
- Add your code in the text box.

Solution 1:

```
<div class="row">  
  <div class="col-xl-6 col-sm-12">50% desktop, 100% mobile</div>  
  <div class="col-xl-6 col-sm-12">50% desktop, 100% mobile</div>  
</div>
```

Solution 2:

```
<div class="row">  
  <div class="col-lg-6 col-sm-12 col-10">Column 1</div>  
  <div class="col-lg-3 col-sm-6 col-10">Column 2</div>  
  <div class="col-lg-3 col-sm-6 col-10">Column 3</div>  
</div>
```

Solution 3:

```
<div class="row">  
  <div class="col-xxl-1 col-xl-2 col-lg-4 col-md-6 col-sm-12">Column 1</div>  
  <div class="col-xxl-11 col-xl-10 col-lg-8 col-md-6 col-sm-12">Column 2</div>  
</div>
```